

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1 (CO1)

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Name: _____ Register No.: _____ Section: _____

Introduction and Problem Formulation

A university stores one record per student, identified by a unique Registration Number (a nine-digit integer in this study). Staff repeatedly ask: *does a record with registration number x exist, and where is it?* The database grows from 1,000 to 1,000,000+ records. This report designs three different search strategies, analyses them mathematically (operation counts, recurrences, asymptotics), implements and measures them on datasets of 1,000, 10,000, 100,000 and 1,000,000 records, recommends a design, and then re-evaluates that design for the changed workload of 50,000 searches per hour on 1,000,000 records with record insertions only once a day.

Notation and cost model

- n = number of records (input size); k = number of searches performed; m = number of buckets of the hash table; $\alpha = n/m$ = load factor; $\lg$ = logarithm to base 2.
- Total cost of a strategy = $P(n) + k \cdot S(n)$, where $P(n)$ is the preprocessing cost (sorting, building a table) and $S(n)$ the cost of one search. Preprocessing is always included, as required by the assignment.
- Costs are measured in two ways: (i) machine-independent counts of the basic operation (key comparisons), and (ii) wall-clock time in a real implementation. Comparing the two is the basis of Task 10.

Summary of results

- Linear search needs about 740,279 comparisons (22.6 ms) per search at $n = 1,000,000$; sorted-array binary search needs about 19.5 comparisons (2.44 μ s); the hash table needs about 1.03 comparisons (0.21 μ s).
- For the original, general design the recommended solution is **sort once + recursive binary search** (preprocessing is repaid after only about 13 searches at $n = 1,000,000$).
- For the changed workload (50,000 searches/hour, daily updates) the design is **modified**: a hash-table index rebuilt once a day becomes the primary lookup structure and the sorted array is kept for ordered/range queries. Under simulated traffic it handles an hour of requests in 65 ms, against 310 ms for binary search and about 20 minutes for linear search.

Task 1 – Design the Algorithm (5 Marks)

Three strategies were chosen so that they differ in **design technique**, in **what they assume about the data**, and in **where the cost is paid** (at search time or in preprocessing). This diversity is what makes the comparison meaningful.

Table 1: The three selected strategies

Approach	Design strategy	Style	Preprocessing	Extra space
A. Linear search	Brute force (sequential scan)	Iterative	None	$\Theta(1)$
B. Binary search	Decrease-and-conquer (decrease by half)	Recursive	Sort the keys (merge sort, divide-and-conquer)	$\Theta(\lg n)$ stack
C. Hash-table search	Hashing: space–time trade-off (input enhancement)	Iterative	Build a hash table with separate chaining	$\Theta(n)$

1.1 Approach A – Linear (Sequential) Search

Basic idea. Examine the registration numbers one by one, from the first record to the last, and stop as soon as the key is found. If the end is reached, the record does not exist. The data may be in any order, so no assumption is made about it.

Design strategy: brute force. **Style:** iterative. **Preprocessing:** none.

Algorithm A: LinearSearch($A[0..n-1]$, key)

```
1. for i ← 0 to n – 1 do
    2. if  $A[i] = \text{key}$  then return i    // record found at position i
3. return –1                          // key absent
```

Justification. It is the natural baseline. It needs no preprocessing and no extra memory, and it is the best choice when the database is tiny, when only one or two searches are made, or when the data cannot be reordered. Every other approach must beat it to justify its preprocessing.

1.2 Approach B – Binary Search on a Sorted Array (Recursive)

Basic idea. If the registration numbers are sorted, compare the key with the middle element. If equal, we are done; if the key is smaller, the record can only lie in the left half; otherwise only in the right half. One half is discarded after every comparison, so the problem size is halved each time.

Design strategy: decrease-and-conquer (decrease by a constant factor of 2). **Style:** recursive. **Preprocessing:** the keys must be sorted once. Merge sort (a divide-and-conquer algorithm) is used for the analysis; the implementation uses Python's built-in Timsort, which is also a $\Theta(n \lg n)$ comparison sort.

Preprocessing: MergeSort(A[lo..hi])

1. **if** $lo \geq hi$ **then return**
2. $mid \leftarrow \lfloor (lo + hi) / 2 \rfloor$
3. MergeSort(A[lo..mid]); MergeSort(A[mid+1..hi])
4. Merge(A[lo..mid], A[mid+1..hi]) // linear-time merge of two sorted halves

Algorithm B: BinarySearch(A, key, lo, hi) // initial call: BinarySearch(A, key, 0, n-1)

1. **if** $lo > hi$ **then return** -1 // base case: empty range
2. $mid \leftarrow \lfloor (lo + hi) / 2 \rfloor$
3. **if** $A[mid] = key$ **then return** mid
4. **else if** $key < A[mid]$ **then return** BinarySearch(A, key, lo, mid - 1)
5. **else return** BinarySearch(A, key, mid + 1, hi)

Justification. Its cost grows with the logarithm of n , so it scales to millions of records with only about 20 comparisons per search. The price is the sorting step, which is why preprocessing must be included in the analysis. It is also the natural recursive algorithm for the recurrence tasks (Tasks 6 and 7).

1.3 Approach C – Hash-Table Search with Separate Chaining

Basic idea. Use a hash function to compute directly, from the registration number, the bucket in which the record should be. Only the (short) list of keys in that bucket has to be scanned. The function used is $h(k) = k \bmod m$, where the number of buckets m is a prime close to $1.3n$ (so the load factor $\alpha = n/m \approx 0.77$). Keys that collide are kept in a linked list (chain) in the same bucket.

Design strategy: hashing, a space–time trade-off (extra memory is spent to make search faster). **Style:** iterative. **Preprocessing:** build the table by inserting all n keys.

Preprocessing: BuildHash(A[0..n-1])

1. $m \leftarrow$ smallest prime $\geq 1.3 \cdot n$
2. **for** $j \leftarrow 0$ **to** $m - 1$ **do** $T[j] \leftarrow$ empty list
3. **for** $i \leftarrow 0$ **to** $n - 1$ **do** append $A[i]$ to the list $T[A[i] \bmod m]$
4. **return** T

Algorithm C: HashSearch(T, key)

1. $j \leftarrow key \bmod m$ // compute the bucket
2. **for** each element x in the list $T[j]$ **do**
 3. **if** $x = key$ **then return** found
4. **return** not found

Justification. It is the only approach whose expected search cost does not depend on n . It is the standard way to implement exact-match lookup on a key, and it is a strong candidate when the number of searches is large relative to the number of updates, as in the Day-4 challenge.

1.4 Why the three approaches are genuinely different

A scans everything and needs no structure; B exploits **order** (it requires sorted data and discards half of the data per step); C exploits **direct addressing by computation** (it requires a hash index and ignores order completely). Their preprocessing costs, space requirements, search costs and worst-case behaviour are all different, and none is a rewrite of another. (An iterative version of binary search would not count as a fourth approach; it is the same algorithm.)

Task 2 – Identify the Basic Operation (2 Marks)

The basic operation is the operation in the innermost, most frequently executed part of the algorithm; all other work (index increments, function-call overhead, computing a bucket number) is executed at most a constant number of times for each execution of the basic operation, so counting it captures the growth of the running time. For all three algorithms the basic operation is a **comparison of the search key with a stored registration number**.

Table 2: Basic operation of each algorithm

Algorithm	Basic operation	Where it occurs	How its count changes with n
A. Linear search	$A[i] = \text{key}$	Body of the single for-loop (line 2)	Executed once per record examined; up to n times, so it grows in direct proportion to n .
B. Binary search	$A[\text{mid}] = \text{key} / \text{key} < A[\text{mid}]$ (counted as one three-way comparison per probe)	Once per recursive call (lines 3–4)	Each call halves the range, so the count grows by one each time n doubles: about $\lg n$.
C. Hash search	$x = \text{key}$	Body of the loop that scans one bucket's chain (line 3)	Equals the chain length scanned, about $1 + \alpha/2$; independent of n as long as α is bounded.

Why it is appropriate. (1) The comparison is performed in every iteration/call, so the total number of executions determines the running time up to a constant factor. (2) It takes constant time that does not depend on n (comparing two integers). (3) It is machine-independent, so it allows the theoretical result to be validated against measured counts in Task 10. For the preprocessing steps the analogous basic operations are the key comparison inside merge sort (for B) and the insertion of a key into a bucket (for C), each of which is $\Theta(1)$ per execution.

Relation to the algorithm's behaviour. In A the comparison count is the number of records scanned, so it follows the position of the key. In B the comparison count is the number of halvings, so it follows the depth of the key in the implicit search tree. In C the comparison count

is the length of one chain, so it follows how evenly the hash function spreads the keys. In the C algorithm the computation of $\text{key mod } m$ occurs exactly once per search and therefore adds only a constant, regardless of n .

Task 3 – Derive the Time-Complexity Function Mathematically (6 Marks)

In every case the input size is n , the number of records. $C(n)$ denotes the number of executions of the basic operation, and the running time is $T(n) \approx c_{\text{op}} \cdot C(n) + d$, where c_{op} is the (constant) cost of one comparison and d covers constant overhead.

3.1 Approach A – Linear search

Successful search. If the key is at position i ($1 \leq i \leq n$), the loop performs exactly i comparisons. If every position is equally likely (probability $1/n$):

$$C_{\text{succ}}(n) = (1/n) \cdot \sum_{i=1}^n i = (1/n) \cdot n(n+1)/2 = (n+1)/2$$

Unsuccessful search. The whole array is scanned: $C_{\text{fail}}(n) = n$. If a fraction p of the searches are successful, the average count is

$$C_{\text{avg}}(n) = p \cdot (n+1)/2 + (1-p) \cdot n$$

For $p = 1/2$ this is $3n/4 + 1/4$, i.e. about 750,000 comparisons for $n = 1,000,000$. Hence $T(n) = c_{\text{op}}(3n/4 + 1/4) + d = \Theta(n)$.

Preprocessing: $P(n) = 0$. **Overall cost for k searches:** $k \cdot C_{\text{avg}}(n) \approx 3kn/4$, which is $\Theta(kn)$; the cost grows linearly in both the database size and the number of searches.

3.2 Approach B – Binary search on a sorted array

Preprocessing (sorting). Merge sort satisfies $S(n) = 2S(n/2) + (n-1)$ with $S(1) = 0$. In the recursion tree there are $\lg n$ levels and the merge work on each level is at most n comparisons, so

$$S(n) \leq n \lg n - n + 1 = \Theta(n \lg n)$$

For $n = 1,000,000$ this is at most about 19 million comparisons. (A comparison-based sort cannot do better than $\Omega(n \lg n)$ in the worst case, so this is the unavoidable price of ordering the data.)

Searching – worst case. After each probe the remaining range has at most $\lfloor m/2 \rfloor$ elements if it had m . After j probes at most $\lfloor n/2^j \rfloor$ elements remain. The range becomes empty when $n/2^j < 1$, i.e. $j > \lg n$. Therefore

$$C_{\text{worst}}(n) = \lfloor \lg n \rfloor + 1$$

Searching – average (successful). Take $n = 2^h - 1$. The decision tree of the probes is a complete binary tree with h levels; level j holds 2^{j-1} keys, each found after j probes. With equally likely keys:

$$C_{\text{succ}}(n) = (1/n) \cdot \sum_{j=1}^h j \cdot 2^{j-1} = ((h-1) \cdot 2^h + 1)/n = ((n+1)/n) \cdot \lg(n+1) - 1 \approx \lg n - 1$$

An unsuccessful search ends at an empty position of the tree and needs about $\lg(n+1)$ probes. For $n = 1,000,000$ these give 18.9 (successful) and 19.9 (unsuccessful) comparisons.

Overall cost for k searches: $T_{\text{total}}(n, k) = S(n) + k \cdot C(n) \approx n \lg n + k \lg n = (n + k) \lg n = \Theta((n + k) \lg n)$. Compared with linear search ($\approx kn/2$ comparisons on average), the preprocessing is repaid when $n \lg n + k \lg n < kn/2$, i.e. $k > n \lg n / (n/2 - \lg n) \approx 2 \lg n$, which is about 40 searches for $n = 1,000,000$ (by comparison counts).

3.3 Approach C – Hash table with separate chaining

Preprocessing (building the table). Creating $m = \lceil 1.3n \rceil$ empty buckets costs $\Theta(m) = \Theta(n)$. Each of the n insertions computes $k \bmod m$ and appends to a list: $\Theta(1)$. Therefore

$$P(n) = \Theta(m) + n \cdot \Theta(1) = \Theta(n)$$

Searching. Assume simple uniform hashing (each key falls into each bucket with probability $1/m$, independently). For a key that is absent, the chain examined contains each of the n stored keys with probability $1/m$, so the expected number of comparisons is $n/m = \alpha$. For a key that is present and was the i -th inserted, the chain contains the keys inserted before it with probability $1/m$ each, so the expected count is $1 + (i - 1)/m$. Averaging over i :

$$C_{\text{succ}} = 1 + (1/n) \cdot \sum_{i=1}^n (i - 1)/m = 1 + (n - 1)/(2m) \approx 1 + \alpha/2, \quad C_{\text{fail}} = \alpha$$

With $\alpha \approx 0.769$ this is ≈ 1.38 comparisons for a hit and ≈ 0.77 for a miss, independent of n . This is a constant, so $S(n) = \Theta(1)$ in expectation. In the worst case all n keys fall into one bucket, giving $C_{\text{worst}}(n) = n$.

Overall cost for k searches: $T_{\text{total}}(n, k) = c_1 \cdot n + k \cdot (1 + \alpha/2) = \Theta(n + k)$ expected. Because preprocessing is linear and each search is constant, this is the cheapest of the three for large k .

Table 3: Summary of derived cost functions

Algorithm	Preprocessing $P(n)$	Search $S(n)$, average	Total for k searches
A. Linear	0	$(n+1)/2$ hit, n miss	$\Theta(kn)$
B. Binary	$\Theta(n \lg n)$	$\approx \lg n - 1$ hit, $\approx \lg n$ miss	$\Theta((n + k) \lg n)$
C. Hash	$\Theta(n)$	$\approx 1 + \alpha/2$ hit, α miss (expected)	$\Theta(n + k)$ expected

Task 4 – Best, Average and Worst-Case Behaviour (4 Marks)

4.1 Linear search

- **Best case:** the requested registration number is the first record. The loop stops after $C = 1$ comparison, which is $\Theta(1)$.
- **Average case:** the key is present and equally likely to be at any position, giving $(n+1)/2$ comparisons. The key is as likely to be near the end as near the start, so on average half the array is scanned. If absent keys are also asked for, the average rises toward n .

- **Worst case:** the key is the last record or is not in the database at all. The loop cannot stop early and performs $C = n$ comparisons. The difference arises only because the algorithm has no information that lets it skip any record.

4.2 Binary search

- **Best case:** the key is exactly the middle element of the initial range; one probe finds it, $C = 1$.
- **Average case:** with the key present and equally likely to be any element, $C \approx \lg n - 1$, because half of all keys lie on the deepest level of the decision tree and a quarter on the level above, so the typical key needs almost the full depth.
- **Worst case:** the key is at a deepest node of the decision tree (for example the smallest or the largest element) or is absent: $C = \lfloor \lg n \rfloor + 1$. The range must be halved until it is empty. Binary search has no input that makes it much slower than this, because the range is halved by every probe regardless of the data.

Preprocessing case analysis. Merge sort performs $\Theta(n \lg n)$ comparisons for every input order, so its best, average and worst cases coincide asymptotically. If the incoming data were already sorted, an adaptive sort such as Timsort would finish in $\Theta(n)$, which is the only input condition that changes the preprocessing cost materially.

4.3 Hash-table search

- **Best case:** the key is the first element of its chain (or, for an absent key, its bucket is empty). One comparison (or none for an empty bucket) is needed.
- **Average case:** keys are spread uniformly by $h(k) = k \bmod m$. A hit needs $\approx 1 + \alpha/2$ comparisons and a miss $\approx \alpha$, both constants for a bounded load factor.
- **Worst case:** all keys land in the same bucket, so the chain has n elements and the search degenerates into linear search with $C = n$. This happens when the hash function interacts badly with the data (for example, if m divides every registration number) or when keys are chosen adversarially. For random 9-digit numbers and a prime m this is vanishingly unlikely: the longest chain observed at $n = 1,000,000$ was only 7.

Measured operation counts for the special cases. Table 4 shows the measured best and worst case comparison counts. For the hash table the worst case was produced deliberately with n keys that are all multiples of m .

Table 4: Measured comparisons in best and worst cases

n	Linear best	Linear worst	Binary best	Binary worst (hit)	Hash best	Hash worst (all collide)
1,000	1	1,000	1	9	1	1,000
10,000	1	10,000	1	13	1	10,000
100,000	1	100,000	1	16	1	100,000
1,000,000	1	1,000,000	1	19	1	1,000,000

The measurements agree with the analysis: linear search and the degenerate hash table reach exactly n comparisons, while binary search reaches 19 comparisons at $n = 1,000,000$, which is within one of $\lfloor \lg 1,000,000 \rfloor + 1 = 20$ (the smallest key happens to lie one level above the deepest level of the probe tree, while an absent key needs the full 20). In time, the worst linear search at $n = 1,000,000$ took 31.7 ms, the worst binary search 2.88 μ s and the all-collision hash search 40.8 ms.

Task 5 – Asymptotic Analysis (3 Marks)

5.1 Notation applied to each algorithm

- **Linear search.** Worst and average case $C(n) = \Theta(n)$: since $n/2 \leq (n+1)/2 \leq n$ for $n \geq 1$, $(n+1)/2 \in \Theta(n)$ with $c_1 = 1/2$, $c_2 = 1$, $n_0 = 1$. Best case $\Omega(1)$ and $O(1)$. Overall: $O(n)$ for any search, $\Omega(1)$ for the fastest; $\Theta(n)$ as the typical and worst-case growth.
- **Binary search.** $C_{\text{worst}}(n) = \lfloor \lg n \rfloor + 1$ satisfies $\lg n \leq \lfloor \lg n \rfloor + 1 \leq 2 \lg n$ for $n \geq 2$, so it is $\Theta(\lg n)$. The best case is $\Theta(1)$. Preprocessing is $\Theta(n \lg n)$. Total: $\Theta((n+k) \lg n)$.
- **Hash table.** Expected search cost $\Theta(1)$ (since $1 \leq 1 + \alpha/2 \leq 2$ for $\alpha \leq 1$), worst case $\Theta(n)$ ($O(n)$ is the correct tight bound for the degenerate input). Building: $\Theta(n)$. Total expected $\Theta(n+k)$.

5.2 Comparison of growth rates

Table 5: Growth of the number of comparisons as n increases from 1,000 to 1,000,000

Algorithm	Growth class	$n = 1,000$	$n = 1,000,000$	Factor for 1000× more data
A. Linear (avg, hit/miss mix)	$\Theta(n)$	742	740,279	×998
B. Binary (avg)	$\Theta(\lg n)$	9.6	19.5	×2.02
C. Hash (avg)	$\Theta(1)$ expected	1.07	1.03	×0.97

The ordering of the growth rates is $1 < \lg n < n$. When the database grows by a factor of 1,000 the work of linear search also grows by about 1,000, the work of binary search merely doubles ($\lg n$ goes from about 10 to about 20), and the expected work of hashing does not change. In the university setting this means that a design that is acceptable at 1,000 records (linear search takes under 25 μ s) becomes unacceptable at 1,000,000 records (more than 20 ms per request), whereas binary and hash search stay in the microsecond range.

Preprocessing grows too. Sorting grows as $\Theta(n \lg n)$, slightly faster than n ; hash-table building grows as $\Theta(n)$. For one-off searching, the preprocessing term dominates ($\Theta(n \lg n) > \Theta(n)$ of linear search), so asymptotic notation on the search function alone would be misleading; the correct comparison is of the totals $\Theta(kn)$ versus $\Theta((n+k) \lg n)$ versus $\Theta(n+k)$. For $k \gg 1$ the totals follow the order $\Theta(n+k) < \Theta((n+k) \lg n) < \Theta(kn)$, which is the order that determines the Task 11 and Task 12 decisions.

Task 6 – Formulate the Recurrence Relation (3 Marks)

The recurrence is derived from the recursive binary-search algorithm of Section 1.2 (Approach B).

- **Input size of the original problem:** $n = hi - lo + 1$, the number of records in the current range.
- **Size of the recursive subproblem:** after comparing with $A[mid]$, the algorithm continues either on $A[lo..mid-1]$ (size $\lfloor (n-1)/2 \rfloor$) or on $A[mid+1..hi]$ (size $\lfloor (n-1)/2 \rfloor \leq \lfloor n/2 \rfloor$). In the worst case the larger half, of size $\lfloor n/2 \rfloor$, is searched.
- **Number of recursive calls:** exactly **one**. The two branches are mutually exclusive (if/else-if), so only one half is ever searched. (Contrast with merge sort, which makes two calls.)
- **Work outside the recursive call:** computing mid and comparing the key with $A[mid]$. This is a constant amount of work, so $f(n) = c$ (one basic operation).
- **Base case:** $n = 0$ ($lo > hi$): the function returns -1 immediately, $T(0) = c_0$, which is constant (no basic operation, $C(0) = 0$). A range of size 1 also terminates after one probe followed by a base-case call.

Putting the components together. The cost of a search over n records equals the cost of the recursive search over at most $\lfloor n/2 \rfloor$ records plus the constant work of one probe. For the number of comparisons in the worst case:

$$C(n) = C(\lfloor n/2 \rfloor) + 1 \quad \text{for } n \geq 1, \quad C(0) = 0$$

and for the running time, $T(n) = T(\lfloor n/2 \rfloor) + c$ for $n \geq 1$ with $T(0) = c_0$. When n is a power of two, the form used for solving is $T(n) = T(n/2) + c$, $T(1) = c$. As a check, the recurrence gives $C(1) = 1$, $C(2) = 2$, $C(3) = 2$, $C(4) = 3$, $C(7) = 3$, $C(8) = 4$, which agrees with the direct count $\lceil \lg n \rceil + 1$ and with the measured worst case of 19 or 20 probes at $n = 10^6$ (Table 4).

For completeness, the preprocessing recurrence. Merge sort splits the array into two halves, makes two recursive calls and merges in linear time: $S(n) = 2S(n/2) + (n - 1)$, $S(1) = 0$ (two calls on size $n/2$, linear work outside the calls).

Task 7 – Solve the Recurrence Relation (4 Marks)

Recurrence to solve: $T(n) = T(n/2) + c$, $T(1) = c$ (n a power of 2; floors only change constants and are handled in 7.3).

7.1 Method 1 – Backward substitution (iteration method)

This method is suitable because the recurrence has a single recursive term and a constant additive term, so repeated expansion shows a clear pattern. Expanding:

$$\begin{aligned} T(n) &= T(n/2) + c \\ &= [T(n/4) + c] + c = T(n/4) + 2c \\ &= [T(n/8) + c] + 2c = T(n/8) + 3c \\ &\vdots \end{aligned}$$

$$= T(n/2^i) + i \cdot c$$

The expansion stops when $n/2^i = 1$, i.e. $i = \lg n$. Substituting gives $T(n) = T(1) + c \cdot \lg n = c + c \lg n$. Hence **$T(n) = \Theta(\lg n)$** .

7.2 Method 2 – Master Theorem

The recurrence has the form $T(n) = aT(n/b) + f(n)$ with $a = 1$, $b = 2$ and $f(n) = c = \Theta(1)$, which satisfies the conditions of the theorem ($a \geq 1$, $b > 1$, $f(n)$ positive). The critical exponent is $\log_b a = \log_2 1 = 0$, so $n^{\log_b a} = n^0 = 1$. Since $f(n) = \Theta(1) = \Theta(n^0)$, this is **Case 2**, and

$$T(n) = \Theta(n^{\log_b a} \cdot \lg n) = \Theta(n^0 \cdot \lg n) = \Theta(\lg n)$$

7.3 Method 3 – Verification by induction (substitution method), including floors

For the exact worst-case count $C(n) = C(\lfloor n/2 \rfloor) + 1$, $C(0) = 0$, claim $C(n) = \lfloor \lg n \rfloor + 1$ for $n \geq 1$. *Base:* $C(1) = C(0) + 1 = 1 = \lfloor \lg 1 \rfloor + 1$. *Step:* let $2^q \leq n < 2^{q+1}$, so $\lfloor \lg n \rfloor = q$. Then $2^{q-1} \leq \lfloor n/2 \rfloor < 2^q$, so by the induction hypothesis $C(\lfloor n/2 \rfloor) = (q-1) + 1 = q$, and $C(n) = q + 1 = \lfloor \lg n \rfloor + 1$. ■ This proves the closed form exactly, including the floor, and confirms the $\Theta(\lg n)$ result for every n , not only powers of two.

7.4 Interpretation

The recursion tree of this algorithm is a single path (one call per level) of depth $\lfloor \lg n \rfloor + 1$, with constant work at each level. The algorithm therefore does work proportional to the **depth**, not to the size of the data: doubling n adds only one extra call. For $n = 1,000,000$ the depth is about 20; for $n = 1,000,000,000$ it would be only about 30. The same depth bounds the extra memory used by the recursion stack, $\Theta(\lg n)$, so recursion is safe in practice (Python's default limit of 1,000 frames is never approached). The cost of the recursive search is negligible compared with the $\Theta(n \lg n)$ sort that makes it possible, which is why preprocessing dominates the cost of a one-off search.

Preprocessing recurrence (for completeness). $S(n) = 2S(n/2) + (n-1)$ has $a = 2$, $b = 2$, $f(n) = \Theta(n) = \Theta(n^{\log_b a})$; this is also Case 2 and gives $S(n) = \Theta(n \lg n)$, consistent with Task 3.

Task 8 – Implement and Test the Algorithms (4 Marks)

All three algorithms, the sorting and hash-building preprocessing, the instrumented (comparison-counting) versions, the test suite and the experiment driver were implemented in Python 3 in a single file, **student_search.py**. The core of the file is reproduced in Appendix A, and the complete file is supplied with this report. The implementations correspond line by line to the pseudocode of Task 1: Approach A is an iterative loop, Approach B is a recursive function with parameters (lo, hi) and base case $lo > hi$, and Approach C computes key mod m and scans one list.

8.1 Datasets

- A master list of 1,000,000 **unique nine-digit registration numbers** was drawn without replacement from 100,000,000–999,999,999 with a fixed random seed (42).

- The datasets of 1,000, 10,000, 100,000 and 1,000,000 records are the **first n elements of the same master list**, so all sizes come from the same distribution and are nested. Each algorithm receives the identical keys in the identical (unsorted) order; algorithm B sorts a copy and algorithm C builds its table from the same list.
- For each n, the same **200 queries** are given to all three algorithms: 100 registration numbers chosen at random from the dataset (successful searches) and 100 numbers that are not in the dataset (unsuccessful searches), with a fixed seed.

8.2 Test cases and correctness checks

- **Edge cases:** an empty database, a one-element database, a two-element database and a small unsorted database, each queried with present keys, a key smaller than every key, a key larger than every key and a key lying between existing keys. All three algorithms were compared with Python's membership test.
- **Randomised cross-validation:** 5,000 records with 500 present and 500 absent queries; for every query the three algorithms had to agree with a set-membership oracle, and for successful searches the returned position had to hold the requested key.
- **Per-dataset check:** before any timing at each n, all 200 queries were verified to give correct results for all three algorithms (assertions in the script). The tests passed for every size.
- **Special cases:** best-case keys (first element, middle element of the sorted array, head of a chain), worst-case keys (last element, smallest element, the final key of a deliberately all-colliding table) and absent keys were executed at every n to produce the data in Task 4.

8.3 Measurement procedure (reproducible)

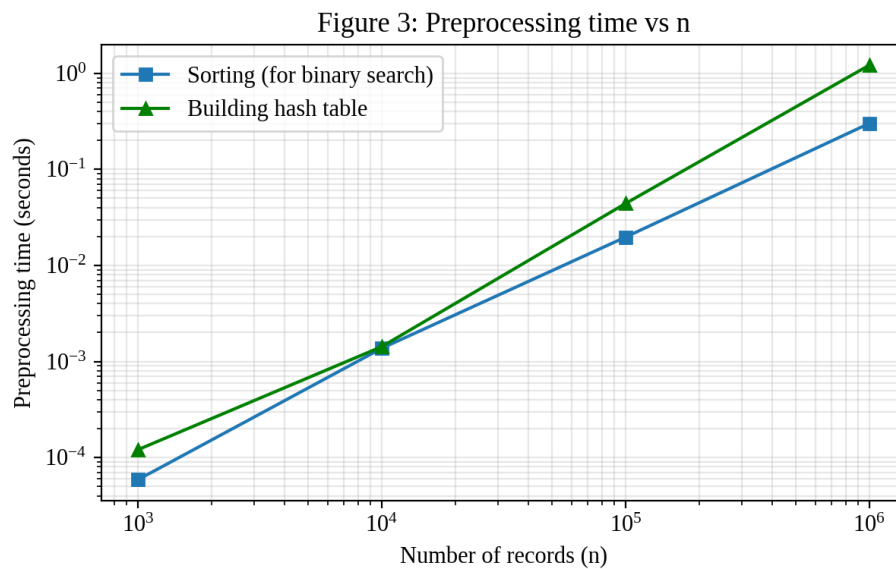
- Time is measured with `time.perf_counter()`; for every measurement the batch of queries is repeated (3 times for linear search at $n \geq 100,000$ and 10 times below; 50 times for the fast algorithms; 3 or 20 times for preprocessing) and the **minimum** is kept to reduce interference from other processes.
- Comparison counts come from separate instrumented copies of the same three functions (a counter incremented once per basic operation), so that counting does not slow down the timed versions.
- One thread, no other deliberate load. Absolute times depend on the machine used (CPython, single core); the **ratios and growth rates** are what the analysis is validated against. To reproduce: run `python student_search.py` (about one minute); results are written to `results.json`.
- For Task 12, a stream of 50,000 shuffled requests (25,000 hits and 25,000 misses) on the 1,000,000-record dataset was run through binary search and hash search. Linear search was timed on a random sample of 300 requests of the same stream and extrapolated to 50,000 requests (running it in full would take about 20 minutes).

Task 9 – Record Experimental Performance (3 Marks)

9.1 Preprocessing

Table 6: Preprocessing time (best of repeated runs) and hash-table statistics

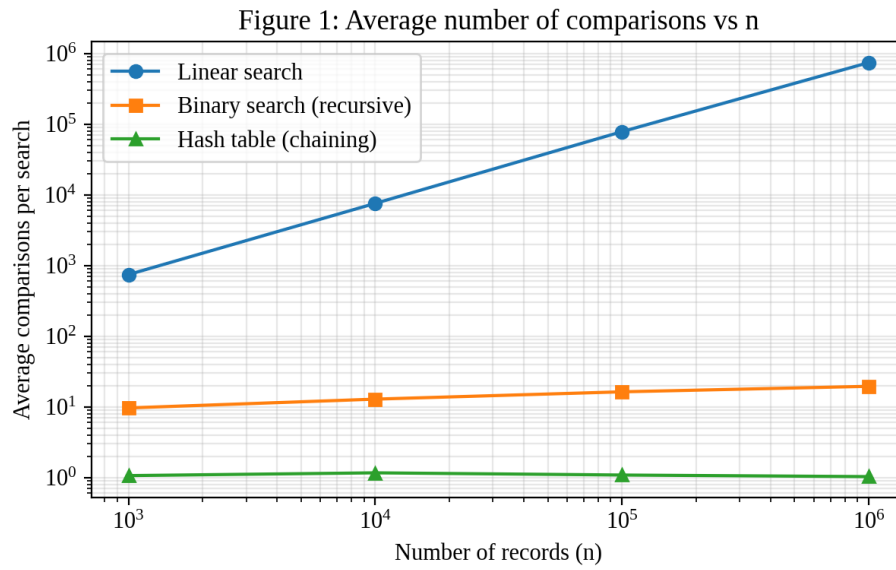
n	Sort (ms)	Build hash table (ms)	Buckets m	Load factor α	Longest chain
1,000	0.059	0.121	1,301	0.769	5
10,000	1.37	1.43	13,001	0.769	6
100,000	19.7	44.1	130,003	0.769	7
1,000,000	301	1,216	1,300,021	0.769	7



9.2 Operation counts

Table 7: Average number of comparisons per search (100 hits and 100 misses per n)

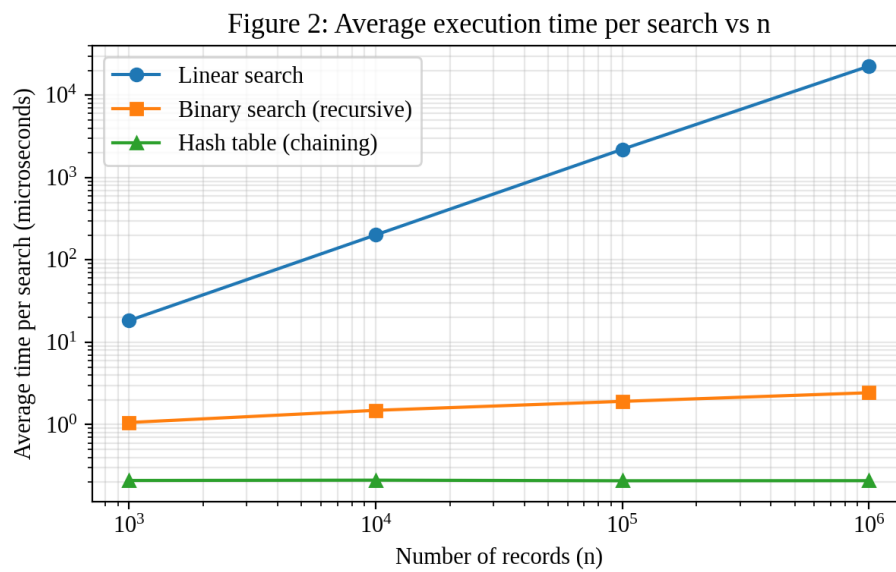
n	Linear hit	Linear miss	Binary hit	Binary miss	Hash hit	Hash miss
1,000	484	1,000	9.29	10.00	1.36	0.77
10,000	5,087	10,000	12.32	13.37	1.46	0.87
100,000	56,124	100,000	15.90	16.71	1.46	0.71
1,000,000	480,558	1,000,000	19.06	19.94	1.32	0.74
Theory, n = 10 ⁶	500,000	1,000,000	18.93	19.93	1.38	0.77



9.3 Execution time

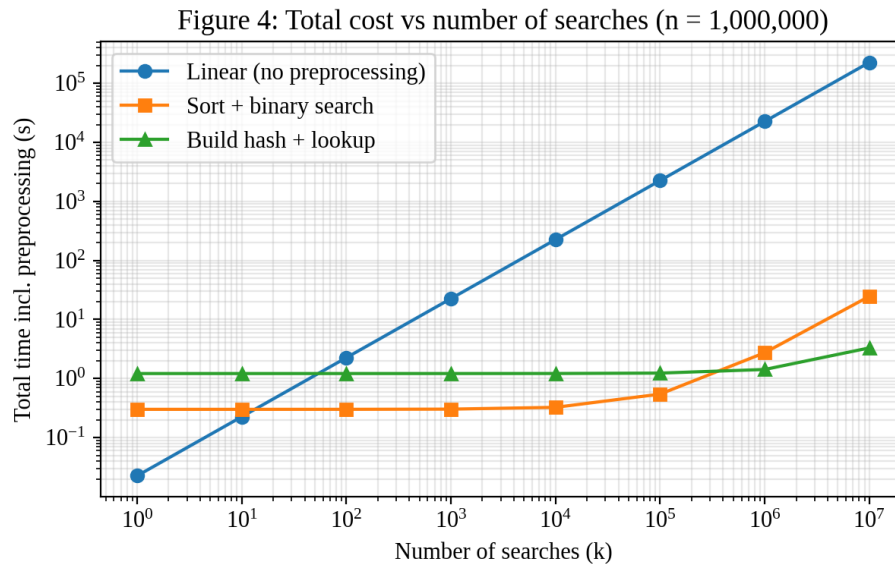
Table 8: Average execution time per search (microseconds, repeated identical batch)

n	Linear hit	Linear miss	Binary hit	Binary miss	Hash hit	Hash miss
1,000	11.6	25.1	1.01	1.11	0.211	0.209
10,000	135	267	1.41	1.57	0.214	0.209
100,000	1,588	2,820	1.84	1.99	0.216	0.200
1,000,000	15,193	29,914	2.34	2.53	0.212	0.206



9.4 Total cost including preprocessing

At $n = 1,000,000$ with the mixed hit/miss workload, the total time for k searches is: linear $0.0226 \cdot k$ seconds; sort + binary $301 \text{ ms} + 2.44 \mu\text{s} \cdot k$; hash table $1,216 \text{ ms} + 0.21 \mu\text{s} \cdot k$. Figure 4 plots these three lines. The break-even points derived from them are: sorting is repaid after $k \approx 13$ searches, building the hash table after $k \approx 54$ searches.



Task 10 – Compare Theory with Experimental Behaviour (4 Marks)

10.1 Do the measurements agree with the theoretical growth?

- **Linear search – agrees.** Theory predicts that comparisons grow in proportion to n . When n grows 1,000-fold the measured average count grows from 742 to 740,279 ($\times 998$). Measured hit counts (480,558 at $n = 10^6$) are within about 4% of the theoretical $(n+1)/2$; the difference is sampling variation of only 100 random queries. Misses are exactly n .
- **Binary search – agrees.** Measured hit counts (19.06 at $n = 10^6$) match the derived formula $(n+1)/n \cdot \lg(n+1) - 1 = 18.93$, and miss counts (19.94) match $\lg(n+1) = 19.93$. The count rises by about 3.3 for each ten-fold increase in n ($9.6 \rightarrow 12.8 \rightarrow 16.3 \rightarrow 19.5$ as a mix), which is the signature of $\Theta(\lg n)$.
- **Hash table – agrees.** Counts remain between 1.03 and 1.17 (mix) at every size, matching the expected $1 + \alpha/2 = 1.38$ for hits and $\alpha = 0.77$ for misses (individual values deviate slightly because only 100 queries of each type were used and because empty-bucket misses cost zero comparisons). Constant behaviour over a 1,000-fold increase confirms $\Theta(1)$ expected search. The longest chain was only 7 at 1,000,000 keys, so the worst case did not occur in practice.
- **Preprocessing.** Sorting went from 0.059 ms to 301 ms, and hash building from 0.121 ms to 1,216 ms. Both rise faster than predicted when n is small and between 100,000 and 1,000,000 (sort $\times 15.3$ against a predicted $\times 12$; hash building $\times 27.6$ against $\times 10$).

10.2 Which algorithm scales better?

Hash search scales best for searching (flat), binary search is a close second (doubling as n grows 1,000-fold) and linear search scales worst ($\times 1,228$ in time). With preprocessing included and enough searches, hashing again has the lowest total, followed by sort + binary search; for a handful of searches only linear search avoids preprocessing (Figure 4).

10.3 Differences between prediction and observation, and likely reasons

- **Time grows slightly faster than counts for linear search** ($\times 1,228$ in time against $\times 998$ in count). A likely reason is memory behaviour: 1,000 integers fit comfortably in the processor cache whereas 1,000,000 Python integers (each a separate object) do not, so each comparison costs a little more at large n .
- **Binary search time rises a little more than $\lg n$ predicts** ($\times 2.30$ in time against $\times 2.02$ in count) for the same cache reason, and each probe also pays the cost of a Python function call, which is much more than a comparison itself.
- **Cache effects also appear in the traffic simulation.** In the Task 12 stream of 50,000 shuffled requests, binary search averaged $6.2 \mu\text{s}$ per search compared with $2.44 \mu\text{s}$ in the repeated 200-query batch of Table 8, since repeated queries keep their probe path in the cache while random ones do not. Hash search similarly rose from 0.21 to $1.29 \mu\text{s}$. Theory counts comparisons and ignores memory hierarchy; this is the main reason why wall-clock numbers differ from count-based predictions.
- **The hash table build is more expensive than theory suggests relative to sorting.** $\Theta(n)$ should beat $\Theta(n \lg n)$, but measured build time ($1,216 \text{ ms}$) is about 4.0 times the sorting time (301 ms) at $n = 1,000,000$. This is largely an implementation effect: the sort is executed by Python's compiled Timsort, whereas the table is built by an interpreted Python loop that allocates about 1.3 million small lists (memory allocation and garbage collection overhead). The comparison counts of the two preprocessing algorithms are not what is being timed here; in a compiled language the linear-time build would be expected to be faster.
- **Absolute counts differ from theory by sampling noise.** Averages from 100 queries have a few per cent of random error; larger samples would converge on the theoretical values.

Overall conclusion of the comparison: the growth rates observed in the experiments (linear, logarithmic, constant) agree with the derived complexities for all three algorithms, and the discrepancies in absolute times can be explained by constant factors, memory effects and the interpreter, none of which changes the order of growth.

Task 11 – Recommend and Defend the Final Design (4 Marks)

11.1 Recommendation

For the **original problem** (a database that grows from 1,000 to over 1,000,000 records, with an unspecified but repeated stream of searches and any preprocessing counted), the recommended

design is: **sort the registration numbers once and answer searches with recursive binary search (Approach B).**

11.2 Evidence and comparison

Table 9: Summary of evidence at $n = 1,000,000$

Criterion	A. Linear	B. Sort + Binary	C. Hash table
Theoretical search cost	$\Theta(n)$	$\Theta(\lg n)$	$\Theta(1)$ expected, $O(n)$ worst
Theoretical preprocessing	None	$\Theta(n \lg n)$	$\Theta(n)$
Measured comparisons / search (mix)	740,279	19.5	1.03
Measured time / search (mix)	22.6 ms	2.44 μ s	0.21 μ s
Measured preprocessing	0	301 ms	1,216 ms
Searches needed to repay preprocessing (vs. linear)	–	≈ 13 (≈ 40 by counts)	≈ 54
Extra memory	None	None (sort in place)	$\approx 1.3n$ buckets
Worst-case guarantee	n comparisons	$\lfloor \lg n \rfloor + 1$, always	n if all keys collide
Range / ordered queries	No (full scan)	Yes (e.g., a batch of numbers)	No

11.3 Why B is preferred over the alternatives

- **Over linear search:** the preprocessing is repaid after about 13 searches at $n = 1,000,000$ (about 40 by comparison counts), after which each search is roughly 9,252 times faster (22.6 ms against 2.44 μ s). Linear search is only appropriate for very small databases or a handful of searches.
- **Over the hash table:** it is only slightly slower per search, but has lower preprocessing time (in our measurement about a quarter of the hash build), needs no additional memory, gives a **guaranteed** worst case of $\lfloor \lg n \rfloor + 1$ comparisons with no dependence on the key distribution or on a hash function, and keeps the records ordered, which supports range queries (for example, all students of one batch) that hashing cannot answer.

11.4 Trade-offs, limitations and scalability

- Preprocessing must be repeated, or the new records merged in ($O(n + b)$ for b new records), when records are added; this is cheap when insertions are rare but would not suit a constantly changing database.
- The sort costs $\Theta(n \lg n)$ (≈ 20 million comparisons at 1,000,000 records) and this cost is independent of how many searches follow; for fewer than about 13 searches linear search is cheaper.
- The solution remains practical as the database grows: going from 1,000 to 1,000,000 records increased the measured search time by only a factor of about 2.3, and 10^9 records would need only about 30 comparisons. Sorting remains the main cost. This conclusion rests on our own measurements (Tables 4–8, Figure 4) and on the derivations in Tasks 3 and 7.

Task 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed requirement: the university now processes 50,000 search requests per hour on a database of 1,000,000 student records, and new records are added only once per day.

A. Impact Analysis (2 Marks)

- **What has changed.** The workload is now known and heavy: 50,000 searches/hour = 13.9 searches per second (one every 72 ms), or about 1,200,000 searches per day, which is more than one search for every record in the database. The database size is fixed at 10^6 , and it is updated in a single daily batch.
- **Assumptions that are affected.** The original design assumed that preprocessing had to be justified by a modest number of searches (break-even after roughly 13 to 40 searches), and it allowed for the database to change. Now (i) the preprocessing cost is spread over about 1.2 million searches per day, so its share per search is negligible; (ii) the per-search cost, not the one-off cost, dominates the daily total.
- **Why the search frequency now matters.** Total cost is $P(n) + k \cdot S(n)$. With $k = 1,200,000$ per day, the term $k \cdot S(n)$ dominates. A per-search cost of 24 ms (linear search) would use about 8 hours of CPU time every day, whereas a few microseconds per search makes it negligible. Even small differences in $S(n)$ are multiplied by k .
- **Why infrequent updates matter.** Because records are added only once a day, the data is **static for 24 hours**. A static structure can be built in a batch outside working hours and used read-only, so there is no need for a structure that supports efficient online insertion (balanced trees, dynamic resizing) and no locking between readers and writers. Preprocessing is therefore a daily cost, not a per-update cost, and any structure that is fast to read is attractive however expensive it is to rebuild (within reason).

B. Evaluation of the Original Design (2 Marks)

- **Behaviour under 50,000 searches/hour.** Sorted array + binary search handled the simulated stream of 50,000 requests on 1,000,000 records in 0.31 s of CPU time (6.2 μ s

per search on shuffled queries), which is 0.0086% of one core. Capacity is not an issue: it could serve about 161,038 searches per second.

- **Is the preprocessing cost acceptable?** Yes. A complete re-sort of 1,002,000 keys took 458 ms (a conservative figure, since merging a sorted batch of new records into the already-sorted array would take only $\Theta(n + b)$). Once a day this is negligible.
- **Could another approach give better overall performance?** Yes. Linear search is clearly unacceptable: the estimated 1,222 s (20 minutes) of CPU per hour is 34% of a core and about 8.1 CPU-hours per day, with ~ 25 ms latency per request. The hash table needed only 0.065 s for the same hour, 4.8 times faster than binary search, and its higher build cost (1.98 s) is repaid after about 310,206 searches, which is 6.2 hours of traffic, so it wins over a day.

C. Adaptation or Redesign (2 Marks)

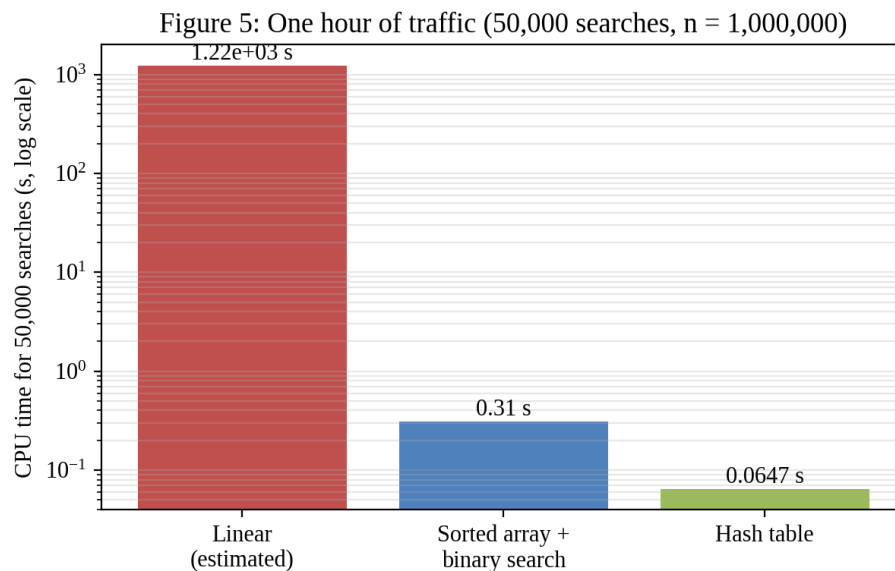
Decision: MODIFY. The core idea of the original design (pay a preprocessing cost once and then search a prepared static structure) is retained; what changes is the structure that is prepared and the way the preprocessing is scheduled.

- **Revised strategy.** A **hash index** (Approach C: key mod m with m a prime $\approx 1.3n$, separate chaining) becomes the primary structure for the exact-match lookups of registration numbers. It is rebuilt **once per day** after the batch of new records has been added; the new table is built on the side and swapped in atomically, so searches are never interrupted. The sorted array is **kept** as a secondary index, maintained by merging each day's new records in $O(n + b)$, and is used for range queries and as a fallback.
- **Guarding the worst case.** Because the table is static for a day, its worst case can be measured at build time: after building, the longest chain is checked (it was 7 at $n = 1,000,000$). If it exceeds a threshold (say 20), the table is rebuilt with a different prime m or a different hash function. This removes the $\Theta(n)$ worst case in practice, which was the main objection to hashing in Task 11.
- **What has changed and its effect on complexity.** Search changes from $\Theta(\lg n)$ (19–20 comparisons) to $\Theta(1)$ expected (about 1.0–1.4 comparisons). Daily preprocessing changes from $\Theta(n \lg n)$ to $\Theta(n)$. The total daily cost changes from $\Theta((n + k) \lg n)$ to $\Theta(n + k)$. The price is about $1.3n$ extra buckets (memory), and no support for ordered queries in the hash index itself, which is why the sorted array is kept.
- **If the traffic ever grows or the database becomes dynamic,** the revised design extends naturally: insertions can be added to the hash table in expected $\Theta(1)$ each (with occasional rebuilding when α grows too large), and for a fixed key set a perfect hash table gives $\Theta(1)$ worst-case search.

D. Evidence-Based Defence (2 Marks)

Table 10: Daily cost for 50,000 searches/hour on 1,000,000 records (1,200,000 searches per day)

Design	Daily rebuild (s)	Per search (μ s)	Searches per day (s)	Total per day (s)
A. Linear	0	24,439	29,326	29,326
B. Sort + binary (original)	0.46	6.21	7.45	7.91
C. Hash index (modified)	1.98	1.29	1.55	3.54



- **Search frequency and dataset size:** with $k = 1,200,000$ searches per day on $n = 1,000,000$ records, per-search cost dominates. Linear search needs about 8.1 CPU-hours per day (Table 10) and is eliminated by the evidence. Binary search and hashing both cost only seconds a day.
- **Preprocessing cost:** the hash build is more expensive than the sort (1.98 s against 0.46 s), but this is a once-a-day cost and is repaid after about 310,000 searches, i.e. in under 7 hours of the day's traffic.
- **Mathematical analysis:** $\Theta(n + k)$ expected for hashing against $\Theta((n + k) \lg n)$ for sorting plus binary search. With $n = 10^6$ and $k = 1.2 \times 10^6$, the ratio of the leading terms is about $\lg n \approx 20$ in comparison counts, and the measured daily ratio is 2.2.
- **Trade-offs, stated honestly:** both binary search and hashing meet the requirement with a very large margin: even binary search would use under 0.01% of one core. The decision to modify the design is therefore based on efficiency and simplicity of operation rather than necessity: the hash index gives the lowest daily total (3.5 s against 7.9 s), constant-time look-ups with lower and more predictable latency, and read-only data that needs no locks,

in exchange for about $1.3n$ additional memory and a worst case that has to be controlled by build-time checks. Keeping the sorted array guarantees that no capability of the original design (ordered access, deterministic worst case, a fallback) is lost.

- **Conclusion of the defence:** the original design is not wrong, but the changed conditions (many reads, one daily write) favour moving the cost from the search to the daily rebuild, and the decision to MODIFY follows directly from the mathematical analysis ($\Theta(n + k) < \Theta((n + k) \lg n) \ll \Theta(kn)$) and from the experiments (Tables 7, 8, 10; Figures 4 and 5).

Practical note. In a real production system the registration-number column of a database would simply carry a primary-key index (usually a B+-tree, which behaves like binary search on a sorted structure with ordered access, or a hash index). The analysis here explains why such an index is worth building and when each type is appropriate.

Overall Conclusion

Linear search, recursive binary search on sorted data and hash-table search have complexities $\Theta(n)$, $\Theta(\lg n)$ and $\Theta(1)$ (expected) per search, with preprocessing of 0, $\Theta(n \lg n)$ and $\Theta(n)$. The recurrence $C(n) = C(\lfloor n/2 \rfloor) + 1$ solves to $\lfloor \lg n \rfloor + 1$ exactly. Experiments on 1,000 to 1,000,000 records confirmed these growth rates; deviations in absolute time were explained by constant factors, cache effects and interpreter overhead. Sort + binary search is the recommended design for the original problem, and the hash-index design, rebuilt daily with a sorted array retained for range queries, is the recommended modification for 50,000 searches per hour with once-a-day updates.

Appendix A – Source Code (core of student_search.py)

The listing below contains the three algorithms, their preprocessing, the instrumented versions used for counting, the query generator and the correctness tests. The remainder of the file (the experiment driver, main()) generates the datasets and the measurements described in Task 8 and is supplied in the accompanying file.

```
"""
```

Student Record Retrieval - three search strategies, experiments for Assignment 1 (DAA).

A. Linear search (iterative, no preprocessing)

B. Binary search (recursive, preprocessing = sort)

C. Hash table search (iterative, preprocessing = build table, separate chaining)

Run: python student_search.py (writes results.json)

```
"""
```

```
import random, time, json, math, sys
```

```
sys.setrecursionlimit(10000)
```

```
SEED = 42
```

```
# ----- Algorithm A: linear search (iterative) -----
```

```
def linear_search(a, key):
```

```
    for i, x in enumerate(a):
```

```
        if x == key:
```

```
            return i
```

```
    return -1
```

```
def linear_search_count(a, key):
```

```
    c = 0
```

```
    for i, x in enumerate(a):
```

```
        c += 1
```

```
        if x == key:
```

```
            return i, c
```

```
    return -1, c
```

```
# ----- Algorithm B: binary search (recursive) -----
```

```
def binary_search_rec(a, key, lo, hi):
```

```
    if lo > hi:                # base case: empty range
```

```
        return -1
```

```
    mid = (lo + hi) // 2
```

```
    if a[mid] == key:
```

```
        return mid
```

```
    if key < a[mid]:
```

```
        return binary_search_rec(a, key, lo, mid - 1)
```

```
    return binary_search_rec(a, key, mid + 1, hi)
```

```

def binary_search_count(a, key, lo, hi, c=0):
    if lo > hi:
        return -1, c
    mid = (lo + hi) // 2
    c += 1          # one three-way comparison per probe
    if a[mid] == key:
        return mid, c
    if key < a[mid]:
        return binary_search_count(a, key, lo, mid - 1, c)
    return binary_search_count(a, key, mid + 1, hi, c)

def preprocess_sort(keys):
    return sorted(keys)          # Timsort: Theta(n log n) comparisons

# ----- Algorithm C: hash table, separate chaining -----
def next_prime(x):
    def is_p(v):
        if v < 2: return False
        i = 2
        while i * i <= v:
            if v % i == 0: return False
            i += 1
        return True
    while not is_p(x): x += 1
    return x

def build_hash(keys, load=1.3):
    m = next_prime(int(len(keys) * load))
    table = [[] for _ in range(m)]
    for k in keys:
        table[k % m].append(k)    # h(k) = k mod m
    return table

def hash_search(table, key):
    chain = table[key % len(table)]
    for i, x in enumerate(chain):
        if x == key:
            return i
    return -1

def hash_search_count(table, key):
    chain = table[key % len(table)]
    c = 0

```

```

    for i, x in enumerate(chain):
        c += 1
        if x == key:
            return i, c
    return -1, c

# ----- helpers -----
def make_queries(keys, n_hit=100, n_miss=100, rng=None):
    hits = rng.sample(keys, n_hit)
    s = set(keys); misses = []
    while len(misses) < n_miss:
        q = rng.randrange(10**8, 10**9)
        if q not in s: misses.append(q)
    return hits, misses

def best_time(fn, reps):
    best = float("inf")
    for _ in range(reps):
        t0 = time.perf_counter(); fn(); t = time.perf_counter() - t0
        best = min(best, t)
    return best

def correctness_tests():
    rng = random.Random(1)
    # edge cases
    for keys in ([], [5], [3, 9], [7, 1, 9, 4, 12, 30, 2]):
        s = sorted(keys); h = build_hash(keys) if keys else [[]]
        probes = keys + [0, 100, 6, 10, 31]
        for q in probes:
            exp = q in keys
            assert (linear_search(keys, q) != -1) == exp
            assert (binary_search_rec(s, q, 0, len(s) - 1) != -1) == exp
            assert (hash_search(h, q) != -1) == exp
    # random
    keys = rng.sample(range(10**8, 10**9), 5000)
    s = sorted(keys); h = build_hash(keys); S = set(keys)
    hits, misses = make_queries(keys, 500, 500, rng)
    for q in hits + misses:
        exp = q in S
        i = linear_search(keys, q); j = binary_search_rec(s, q, 0, len(s) - 1); k =
        hash_search(h, q)
        assert (i != -1) == exp and (j != -1) == exp and (k != -1) == exp
        if exp: assert keys[i] == q and s[j] == q
    return True

```

